The Journal of Open Source Software

quantum_al: correlation-aware active learning for materials discovery

Arnav Kapoor [1]

1 Indian Institute of Science Education and Research Bhopal, India

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 07 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

A single density-functional-theory (DFT) calculation returns many properties of a material at once (band gap, formation energy, magnetic moment), all derived from the same electronic structure and therefore correlated. Active learning chooses which material to calculate or synthesize next under a fixed budget (Lookman et al., 2019). Whether that choice should account for the correlation between properties, and when doing so can help, is an open question. `quantum_al` is a Python package for answering it rigorously. It provides (1) multi-property acquisition criteria derived from Bayesian optimal experimental design (Chaloner & Verdinelli, 1995; Lindley, 1956): the joint expected information gain (EIG), its correlation-blind counterpart, the exact total-correlation term (Watanabe, 1960) that separates them, unit-invariant A- and E-optimal analogues, and a greedy batch EIG with a $(1-1/e)$ optimality guarantee (Krause & Guestrin, 2005; Nemhauser et al., 1978); (2) diagnostics that explain why two criteria do or do not lead to different outcomes; (3) a synthetic testbed with tunable cross-property correlation and noise; (4) a benchmark harness on real Materials Project data (Jain et al., 2013) with nine classical baselines and paired statistics with multiple-comparison correction (Holm, 1979); and (5) a quantum-inspired covariance formalism with a Qiskit (Javadi-Abhari et al., 2024) circuit realization, kept as an independently evaluated alternative construction.

Statement of need

Active learning is now a standard way to reduce the cost of materials discovery (Lookman et al., 2019; Pyzer-Knapp et al., 2022; Vandermause et al., 2020), and multi-task models that share information across properties are well established (Zhang & Yang, 2022). Proposals to make the *acquisition* step itself correlation-aware, including quantum and quantum-inspired constructions (Biamonte et al., 2017; Schuld & Petruccione, 2021), are harder to assess. Evaluating one honestly requires the criterion and its correlation-blind ablation behind the same interface, real multi-property data in which each query reveals all properties, strong classical baselines, paired trials with corrected statistics, and a way to tell whether a null result means "the idea does not work" or "the experiment could not detect it". `quantum_al` packages all of these.

Its central contribution is the link between theory and diagnostics. The documentation proves that the joint EIG equals the sum of per-property EIGs minus a total-correlation term, that this term is non-negative (Hadamard's inequality), and that it is *second order* in the ratio of epistemic to aleatoric variance while the marginal terms are first order. The joint and correlation-blind criteria can therefore rank two candidates differently only when their marginal scores are nearly tied. `quantum_al.diagnostics` measures exactly these quantities on a candidate pool (the overlap of the two criteria's top batches, the spread of the correlation term relative to the marginal scores, the median signal-to-noise ratio), which turns a null benchmark result into a mechanistic explanation. The package serves materials-informatics researchers

DRAFT

comparing acquisition strategies, including their own, on real data; researchers in Bayesian experimental design who want a tested, documented multi-output EIG implementation; and quantum-computing researchers who want a worked example of taking a quantum-inspired construction to a verified circuit.

State of the field

General-purpose active-learning libraries such as modAL (Danka & Horvath, 2018) and ALiPy (Tang et al., 2019) provide many classical single-target strategies behind a scikit-learn-compatible interface (Pedregosa et al., 2011), but they include no multi-property acquisition criteria and no materials data. BoTorch (Balandat et al., 2020) implements Monte-Carlo acquisition functions for Bayesian optimization, including multi-output models, but it targets optimization rather than learning a predictive model over a pool, and it is built around Gaussian-process surrogates rather than the tree ensembles common in materials informatics. Materials benchmarks such as Matbench (Dunn et al., 2020), the sequential-learning benchmark of Rohr et al. (2020) and Olympus (Häse et al., 2021) standardize datasets and protocols but do not provide or analyze multi-property acquisition criteria. BatchBALD (Kirsch et al., 2019) and BALD (Houlsby et al., 2011) are the closest information-theoretic methods, for classification. `quantum_al` contributes the Gaussian, multi-output, ensemble-based analogue for regression, with proofs of its properties, the diagnostics that follow from them, and a statistically careful real-data benchmark, rather than reimplementing general active-learning infrastructure that these packages already provide.

Software design

Every acquisition strategy implements one method, `select_next_experiments(X_candidates, X_train, Y_train, n_select)`, and is run by the shared pool-based loop `quantum_al.loop.run_active_` so a new criterion or baseline never requires changes to the benchmark harness. `quantum_al.acquisition` computes all criteria from two quantities that a random-forest ensemble (Breiman, 2001) supplies for each candidate: the $K \times K$ covariance of per-tree predictions and the per-property out-of-bag residual variance. Batch selection uses the cross-candidate covariance through Sylvester's determinant identity, at a cost independent of the batch size. The test suite checks each criterion against its own mathematical properties: the decomposition identity, the sandwich bound $\max_k \mathrm{EIG}_k \le \mathrm{EIG} \le \sum_k \mathrm{EIG}_k$, exact invariance to changing units, the tight two-property bound on the correlation term, and monotonicity, submodularity and the $(1 - 1/e)$ bound of the greedy batch by brute-force enumeration on small pools. `quantum_al.stats` reports paired t and Wilcoxon tests, bootstrap confidence intervals, effect sizes and Holm correction. The tests and the circuit benchmark check that the Qiskit circuit in `quantum_al.circuit` reproduces the classical closed form before any resource estimate is reported. Core dependencies are NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020) and scikit-learn; Qiskit and pymatgen (Ong et al., 2013) are optional extras.

One design rule applies throughout: the benchmark scripts raise an error instead of substituting illustrative numbers when a computation fails, and every number in the documentation is written to `results/` by a script in `benchmarks/`. The main trade-off is the narrow selector interface, a scored ranking over a candidate pool given a labeled set. Multi-fidelity and cost-aware protocols are out of scope for now.

A minimal example, which needs no data download:

```
from sklearn.model_selection import train_test_split
from quantum_al import EnsembleCriterionSelector, make_correlated_tasks, run_active_lear
from quantum_al.data_utils import standardize
```

DRAFT

```
X, Y, info = make_correlated_tasks(n_samples=300, correlation=0.9, noise=0.3, seed=0)
X_pool, X_test, Y_pool, Y_test = train_test_split(X, Y, test_size=0.3, random_state=0)
X_pool, X_test = standardize(X_pool, X_test)

for criterion in ["eig", "marginal_eig"]:
    selector = EnsembleCriterionSelector(criterion, n_estimators=50, seed=0)
    curve = run_active_learning(selector, X_pool, Y_pool, X_test, Y_test,
                                n_initial=20, n_rounds=5, batch_size=8, seed=0)
    print(criterion, curve.aulc())
```

Research impact statement

The author has used `quantum_al` for two empirical studies, reported in a manuscript in preparation (Kapoor, 2026). On four real Materials Project property pairs, the joint EIG never significantly outperformed its correlation-blind ablation, and it beat random selection significantly on one pair. The quantum-inspired formalism did not outperform nine classical baselines on five real regression tasks, and ablations attributed a later improvement entirely to its use of ensemble disagreement rather than to the quantum-specific components. The package's diagnostics explain the first result. On real candidate pools the correlation term varied only 3 to 9% as much as the marginal scores across candidates, so the joint and correlation-blind criteria ranked candidates almost identically (Spearman correlation of at least 0.98) and shared most of every batch, as the theory predicts for a second-order term. In a controlled synthetic study that varied cross-property correlation (0 to 0.99) and noise independently, the correlation term grew with correlation as predicted, but no difference between the two criteria was significant after correction in any of eight settings. These results are negative, and the package is built so that they are reproducible and checkable: every number is regenerated by a script from stored inputs.

The software has not yet been adopted outside the author's own work. Its near-term value is reuse. A researcher with a new correlation-aware criterion can implement one method and test it against the same baselines, data, statistics and diagnostics, and can use the synthetic testbed to find out in advance whether their benchmark has the power to detect the effect they expect.

AI usage disclosure

Generative AI (Claude, Anthropic, several model versions across development sessions in 2025 and 2026) was used extensively: to write and refactor most of the Python code in `src/quantum_al/` and `benchmarks/`, to draft the test suite and documentation, including the derivations in the theory page, and to draft this paper and the accompanying manuscripts. AI assistance also fact-checked an earlier version of this project by executing its code, which showed that an earlier evaluation had reported numbers that did not reproduce. The author directed the work throughout: framing the research questions, choosing which experiments and fixes to pursue, requiring that every reported result come from an actual run of the code, rejecting drafts and results that did not meet this standard, and reviewing the final code, tests, derivations, statistical methodology and reported findings. The author takes full responsibility for the correctness, originality and reporting of everything in this repository and the accompanying manuscripts.

Acknowledgements

This work uses data from the Materials Project (Jain et al., 2013).

DRAFT

References

Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E. (2020). BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. *Advances in Neural Information Processing Systems*, *33*. https://doi.org/10.48550/arXiv.1910.06403

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). Quantum machine learning. *Nature*, *549*, 195–202. https://doi.org/10.1038/nature23474

Breiman, L. (2001). Random forests. *Machine Learning*, *45*, 5–32. https://doi.org/10.1023/A:1010933404324

Chaloner, K., & Verdinelli, I. (1995). Bayesian experimental design: A review. *Statistical Science*, *10*(3), 273–304. https://doi.org/10.1214/ss/1177009939

Danka, T., & Horvath, P. (2018). ModAL: A modular active learning framework for Python. *arXiv Preprint arXiv:1805.00979*. https://doi.org/10.48550/arXiv.1805.00979

Dunn, A., Wang, Q., Ganose, A., Dopp, D., & Jain, A. (2020). Benchmarking materials property prediction methods: The Matbench test set and Automatminer reference algorithm. *Npj Computational Materials*, *6*, 138. https://doi.org/10.1038/s41524-020-00406-3

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Häse, F., Aldeghi, M., Hickman, R. J., Roch, L. M., Christensen, M., Liles, E., Hein, J. E., & Aspuru-Guzik, A. (2021). Olympus: A benchmarking framework for noisy optimization and experiment planning. *Machine Learning: Science and Technology*, *2*(3), 035021. https://doi.org/10.1088/2632-2153/abedc8

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70.

Houlsby, N., Huszár, F., Ghahramani, Z., & Lengyel, M. (2011). Bayesian active learning for classification and preference learning. *arXiv Preprint arXiv:1112.5745*. https://doi.org/10.48550/arXiv.1112.5745

Jain, A., Ong, S. P., Hautier, G., Chen, W., Richards, W. D., Dacek, S., Cholia, S., Gunter, D., Skinner, D., Ceder, G., & Persson, K. A. (2013). Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL Materials*, *1*(1), 011002. https://doi.org/10.1063/1.4812323

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., & Gambetta, J. M. (2024). Quantum computing with Qiskit. *arXiv Preprint arXiv:2405.08810*. https://doi.org/10.48550/arXiv.2405.08810

Kapoor, A. (2026). *Correlation-aware information gain does not outperform ensemble disagreement in materials active learning*.

Kirsch, A., Amersfoort, J. van, & Gal, Y. (2019). BatchBALD: Efficient and diverse batch acquisition for deep Bayesian active learning. *Advances in Neural Information Processing Systems*, *32*. https://doi.org/10.48550/arXiv.1906.08158

Krause, A., & Guestrin, C. (2005). Near-optimal nonmyopic value of information in graphical models. *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, 324–331. https://doi.org/10.48550/arXiv.1207.1394

Lindley, D. V. (1956). On a measure of the information provided by an experiment. *The Annals*

DRAFT

of Mathematical Statistics, *27*(4), 986–1005. https://doi.org/10.1214/aoms/1177728069

Lookman, T., Balachandran, P. V., Xue, D., & Yuan, R. (2019). Active learning in materials science with emphasis on adaptive sampling using uncertainties for targeted design. *Npj Computational Materials*, *5*, 21. https://doi.org/10.1038/s41524-019-0153-8

Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, *14*, 265–294. https://doi.org/10.1007/BF01588971

Ong, S. P., Richards, W. D., Jain, A., Hautier, G., Kocher, M., Cholia, S., Gunter, D., Chevrier, V. L., Persson, K. A., & Ceder, G. (2013). Python materials genomics (pymatgen): A robust, open-source python library for materials analysis. *Computational Materials Science*, *68*, 314–319. https://doi.org/10.1016/j.commatsci.2012.10.028

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Pyzer-Knapp, E. O., Pitera, J. W., Staar, P. W. J., Takeda, S., Laino, T., Sanders, D. P., Sexton, J., Smith, J. R., & Curioni, A. (2022). Accelerating materials discovery using artificial intelligence, high performance computing and robotics. *Npj Computational Materials*, *8*, 84. https://doi.org/10.1038/s41524-022-00765-z

Rohr, B., Stein, H. S., Guevarra, D., Wang, Y., Haber, J. A., Aykol, M., Suram, S. K., & Gregoire, J. M. (2020). Benchmarking the acceleration of materials discovery by sequential learning. *Chemical Science*, *11*, 2696–2706. https://doi.org/10.1039/C9SC05999G

Schuld, M., & Petruccione, F. (2021). *Machine learning with quantum computers* (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-83098-4

Tang, Y.-P., Li, G.-X., & Huang, S.-J. (2019). ALiPy: Active learning in Python. *arXiv Preprint arXiv:1901.03802*. https://doi.org/10.48550/arXiv.1901.03802

Vandermause, J., Torrisi, S. B., Batzner, S., Xie, Y., Sun, L., Kolpak, A. M., & Kozinsky, B. (2020). On-the-fly active learning of interpretable Bayesian force fields for atomistic rare events. *Npj Computational Materials*, *6*, 20. https://doi.org/10.1038/s41524-020-0283-z

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Watanabe, S. (1960). Information theoretical analysis of multivariate correlation. *IBM Journal of Research and Development*, *4*(1), 66–82. https://doi.org/10.1147/rd.41.0066

Zhang, Y., & Yang, Q. (2022). A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering*, *34*(12), 5586–5609. https://doi.org/10.1109/TKDE.2021.3070203

DRAFT